Lab 2 - Replicated Counter Service

Student: Ongarbekov Ilyas | Course: Distributed Systems | Test Report

1. What I built

In this lab I made a counter service using Python, gRPC and Protocol Buffers. The client can increase a counter or read its value. I added retry handling, idempotency keys, a lock for concurrent requests, and Lamport logical clocks. I also started three independent replicas on ports 50051, 50052 and 50053 and tested a majority acknowledgement rule (2 out of 3).

For recovery, I implemented a simple Restore RPC. I select a source replica and copy its counter value to the other replicas. This is manual state copying, not a complete automatic recovery protocol.

2. Requirements and tests

Task	What I checked	Evidence
A2 - RPC and retry	Increment, Get, duplicate keys, timeout retry	test-counter.py; 4 relevant tests
A3 - Concurrency	2,000 increments give the exact expected result	test_concurrent_increments_exact
B1 - Logical clocks	Client/server event timestamps increase correctly	test-lamport.py; JSONL traces
C1 - Quorum	Three, two, or one responding replica	test-failures.py; 3 passed
Recovery (extra)	Restore the value to other replicas	test-recovery-sync.py; passed

3. Test environment

Operating system: macOS (Apple Silicon Mac). Python: 3.14.7, as shown by the VS Code environment during testing. Packages used: grpcio, grpcio-tools, protobuf and pytest. The exact installed package versions must be attached from **python3 -m pip freeze**; I did not record them in the available test output.

4. Test cases: expected vs actual

ID	Test	Expected result	Actual result
T1	Increment by delta	Correct new value	PASS
T2	Duplicate key	No second increment	PASS
T3	Concurrent 2,000 writes	Exact total of 2,000	PASS
T4	Get missing counter	Found = false	PASS
T5	Retry after timeout	Applied only once	PASS
T6	3 replicas online	Quorum succeeds	PASS
T7	1 replica offline	2 acknowledgements; success	PASS
T8	2 replicas offline	Quorum fails	PASS

Automated pytest result: **8 passed in 0.80 seconds**. A separate synchronization test printed RECOVERY TEST PASSED.

5. Failure injection

I simulated unavailable replicas by using unreachable ports. When all three replicas replied, the operation succeeded. When one replica was unavailable, two replies were enough for quorum. When two replicas were unavailable, one reply was not enough, so the manager returned failure.

Scenario	Injected fault	Expected invariant	Observed
All online	None	At least 2 ACKs	PASS (3/3)
One offline	One unreachable port	At least 2 ACKs	PASS (2/3)
Two offline	Two unreachable ports	No commit reported	PASS (1/3)
Retry	Delayed response	Duplicate not applied twice	PASS

These tests check acknowledgement counting and retry behavior. They do not prove that every committed value survives a restart. My replicas currently keep state only in memory.

6. Performance evaluation (Task C4)

I ran 2,000 Increment requests per configuration and measured latency in milliseconds. Median is the middle latency; p95 is the value below which about 95% of requests finish.

Configuration	Median (ms)	p95 (ms)	Requests
Single replica, 1 client	0.745	0.890	2,000
Single replica, 16 clients	3.800	5.669	2,000
Quorum (3 replicas), 1 client	2.710	5.273	2,000
Quorum (3 replicas), 16 clients	10.819	18.452	2,000

The quorum path is slower because it sends a request to three servers instead of one. My implementation contacts the replicas sequentially, which adds extra waiting time. More concurrent clients also increase latency because the servers and Python threads share resources. For this small counter service, the extra delay may be acceptable when availability matters, but this test does not prove strong consistency. The measurements also include the cost of opening gRPC channels.

7. Lamport clocks and event ordering (Task B2)

I ran three clients: client-1 incremented lamport_x twice, client-2 incremented lamport_y twice, and client-3 issued two Get requests. Each client used its own Lamport clock. The server recorded RECV, APPLY and SEND events.

First causal chain: client-1 SEND Increment (L=1) -> server RECV (L=2) -> APPLY (L=3) -> SEND reply (L=4) -> client-1 RECV reply (L=5). Each step is ordered by a message edge or local program order.

Second causal chain: client-1 RECV first reply (L=5) -> client-1 SEND second Increment (L=6) -> server RECV (L=10) -> APPLY (L=11) -> SEND (L=12) -> client-1 RECV (L=13). The second send cannot happen before that client receives its first reply.

The initial SEND events of client-1, client-2 and client-3 each had L=1. There was no communication between those initial client sends, so they were concurrent in the happened-before sense. Equal timestamps do not mean the events happened at the same physical time. Also, different Lamport values alone do not prove causality; vector clocks can distinguish concurrent events.

8. Reflection questions

Q1. Why is a timeout ambiguous, and how do retries stay safe? After a timeout, the client does not know whether the request was lost or whether the server executed it but the reply was lost. I reuse the same idempotency key for retries, so the server can recognize a duplicate and avoid incrementing twice.

Q2. Why does a smaller Lamport value not prove causality? Lamport clocks only guarantee that if A happened-before B, then L(A) is smaller than L(B). Two unrelated events can have different timestamps without a causal connection. For example, separate clients can perform operations without communicating.

Q3. What failures can a three-replica majority tolerate? A majority needs two acknowledgements. One unavailable replica still leaves two available replicas, so a write can be acknowledged. If two replicas are unavailable, a majority is impossible and the service must not report success. In this prototype, this is an acknowledgement/liveness check; committed data durability and full safety are not guaranteed.

Q4. What is the smallest change needed to survive restarts? I would save the counter state and processed idempotency keys to durable storage and reload them after restart. This is related to persistent logs and state recovery used by more advanced replication protocols.

9. Known limitations

First, all replicas store counters in memory, so restarting them loses their values. Second, recovery needs a manually chosen source and does not copy idempotency-key history. Third, the quorum manager counts acknowledgements but does not implement a consensus protocol or guarantee strong consistency under concurrent writes. Fourth, my server log does not include request IDs, which makes tracing each request across clients and replicas harder.

10. Conclusion

This lab helped me understand how gRPC communication, retries, locks, Lamport clocks and quorum checks work. My eight automated tests passed, and I measured the performance of single-replica and three-replica requests. I also copied a counter value to other replicas and checked that they matched. The project is a working learning prototype, but it still needs durable storage and safer replica synchronization.

Appendix A - Example event trace

Captured client/server events from the Lamport test (selected excerpt):

client-1 SEND Increment L=1
server RECV lamport_x L=2
server APPLY lamport_x L=3
server SEND lamport_x L=4
client-1 RECV IncrementReply L=5
client-1 SEND Increment L=6
server RECV lamport_y L=7
server APPLY lamport_y L=8
server SEND lamport_y L=9
server RECV lamport_x L=10
server APPLY lamport_x L=11
server SEND lamport_x L=12
client-1 RECV IncrementReply L=13
server RECV lamport_x L=13
server SEND lamport_x L=14

server RECV lamport_y L=15

server APPLY lamport_y L=16

server SEND lamport_y L=17

Appendix B - Commands and reproducibility

The project contains counter.proto, server.py, client.py, clocks.py, replication.py, recovery.py, tests and README.md. Example commands from the Lab2 folder:

python3 -m pip install -r requirements.txt

python3 -m grpc_tools.protoc -I. --python_out=. --grpc_python_out=. counter.proto

python3 server.py --port 50051

python3 server.py --port 50052

python3 server.py --port 50053

python3 -m pytest tests/test-counter.py tests/test-failures.py -v

python3 perf-benchmark.py

python3 test-lamport.py

python3 test-recovery-sync.py

python3 check-consistency.py

python3 -m pip freeze > pip-freeze.txt

Run the three server commands in separate terminals. The benchmark script in my current folder is named perf-benchmark.py. A full package-version appendix is still needed: paste the actual output of pip freeze into the submission.

Appendix C - Additional evidence to include before submission

Attach a screenshot or saved terminal output showing 8 passed, the four benchmark results, the recovery result and the relevant JSONL event log excerpt. The source code and this report should be submitted together. This report does not claim tests that were not observed.